

Temporización No Bloqueante con millis()

Reporte de práctica — Parte 2 — Sistemas Programables

1. Descripción del proyecto

Esta segunda parte de la práctica retoma el circuito de tres LEDs utilizado en la Parte 1, pero sustituye por completo el uso de `delay()` por una estrategia de temporización basada en la función `millis()`. El objetivo es lograr que los tres LEDs parpadeen de forma verdaderamente independiente, cada uno con su propio intervalo (500 ms, 1000 ms y 1500 ms), sin que uno bloquee la ejecución del otro.

Como reto adicional, se incorporó una cuarta tarea totalmente independiente: la impresión del mensaje "Hola Mundo" por el Monitor Serial cada 3000 ms, demostrando que el enfoque no bloqueante permite añadir nuevas tareas sin afectar el comportamiento de las ya existentes.

2. Objetivos de aprendizaje

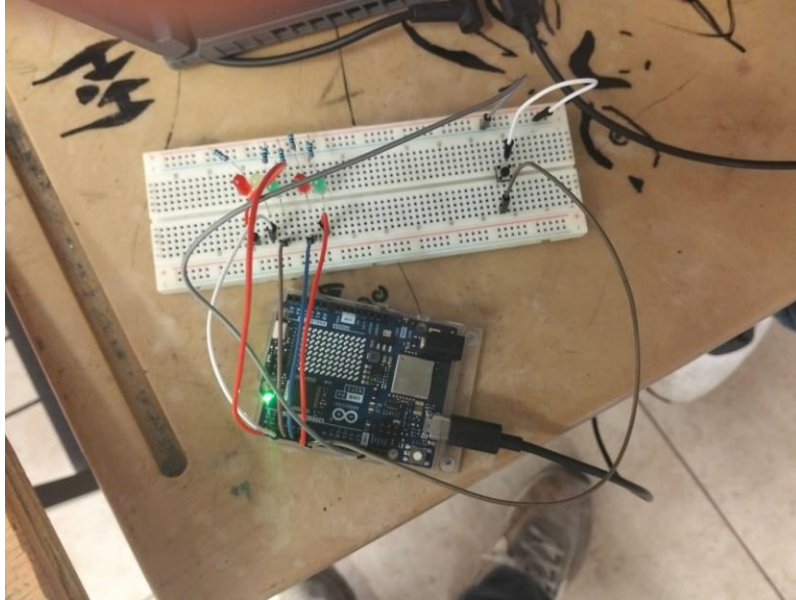
- Comprender el funcionamiento de la función `millis()` como temporizador no bloqueante.
- Implementar múltiples tareas independientes dentro de un mismo `loop()` sin uso de `delay()`.
- Aplicar el patrón de comparación de tiempos (ahora - previo $\geq$ intervalo) para controlar eventos periódicos.
- Verificar que las tareas se ejecutan de forma concurrente y no secuencial, a diferencia del antipatrón de la Parte 1.
- Extender el sistema agregando una tarea adicional (mensaje serial) sin modificar la lógica de los LEDs.

3. Material utilizado

- Arduino UNO R4 WiFi
- Protoboard
- 3 LEDs de colores distintos
- 3 resistencias limitadoras de corriente
- Cables de conexión (jumpers)
- Cable USB para alimentación y programación

4. Diagrama de conexión

El montaje físico es idéntico al de la Parte 1: los LEDs permanecen conectados a los pines digitales 2, 3 y 4, cada uno en serie con su resistencia limitadora hacia GND. No fue necesario modificar el circuito, únicamente el código.



5. Análisis del código

5.1 Variables de estado y de tiempo

Para cada LED se definen dos variables: una booleana que guarda su estado actual (encendido o apagado) y una marca de tiempo (previoLED) que almacena el momento del último cambio. Adicionalmente, se define una constante con el intervalo deseado para cada tarea.

5.2 El patrón de comparación con millis()

```
if (ahora - previoLED1 >= INTERVALO_LED1) {  
    previoLED1 = ahora;  
    estadoLED1 = !estadoLED1;  
    digitalWrite(LED1, estadoLED1);  
}
```

En cada vuelta del loop(), el programa obtiene el tiempo actual con millis() y lo compara contra la última vez que cada tarea se ejecutó. Si ya transcurrió el intervalo correspondiente, se actualiza la marca de tiempo y se invierte el estado del LED con el operador de negación (!). A diferencia de delay(), esta comparación no detiene la ejecución: el programa sigue evaluando las demás condiciones en la misma vuelta del ciclo.

5.3 Tarea adicional independiente

Se agregó una cuarta tarea que sigue exactamente el mismo patrón, pero en lugar de controlar un LED, imprime "Hola Mundo" por el Monitor Serial cada 3000 ms. Esta tarea es completamente independiente de las demás: no comparte variables ni depende del estado de los LEDs, lo cual demuestra la escalabilidad del enfoque con millis().

6. Comparación con la Parte 1

A diferencia del antipatrón con delay(), en este programa el microcontrolador nunca se detiene por completo. En cada vuelta del loop() se revisan las cuatro condiciones de tiempo, y solo se ejecuta la acción correspondiente cuando el intervalo se ha cumplido. Esto permite que los tres LEDs parpadeen genuinamente de forma simultánea e independiente, y que la tarea del mensaje serial ocurra en paralelo sin ninguna interferencia.

7. Conclusiones

Esta práctica permitió comprender de forma directa por qué `millis()` es la alternativa recomendada frente a `delay()` cuando un programa necesita manejar múltiples tareas con temporizaciones distintas. El patrón de comparación de tiempos resultó sencillo de replicar para cada nueva tarea, lo que evidencia su escalabilidad: agregar el mensaje serial cada 3 segundos no requirió modificar ni afectar el comportamiento de los LEDs ya existentes. Este enfoque no bloqueante es la base de programas más complejos que deben responder a sensores, comunicación en red o entradas del usuario mientras controlan actuadores en tiempo real.

8. Video del funcionamiento

Se anexa video demostrativo mostrando el parpadeo simultáneo e independiente de los tres LEDs, seguido de la captura del Monitor Serial imprimiendo "Hola Mundo" cada 3 segundos.